

Annapurna Modern Data Stack

Data Ingestion, Storage, Transformation, Analytics and Validation Report

Project	Annapurna Data Pipeline
Database	PostgreSQL 16
Object Storage	MinIO
Analytics Engine	DuckDB
Processing	Python and Pandas
Output Formats	CSV and Parquet

This report documents the completed local modern data stack implementation, generated datasets, database validation, object storage validation and finance reconciliation results.

1. Executive Summary

The Annapurna data pipeline was executed successfully. The workflow processed raw sales data, cleaned and deduplicated records, generated analytical outputs, created Parquet datasets, exported dimensional data to PostgreSQL, created a DuckDB database, uploaded project outputs to MinIO and generated a finance reconciliation report.

Metric	Result
Raw rows processed	1,137,585
Duplicate records removed	16,661
PostgreSQL stores	12
PostgreSQL product categories	14
PostgreSQL products	1,224
PostgreSQL price revisions	4,320
MinIO objects	172
MinIO storage shown	141.7 MiB
Finance reconciliation	Completed

2. Objective

The objective was to build a local modern data stack for the Annapurna project. The stack separates data processing, object storage, relational storage, analytical querying, dimensional modeling and reporting. The final outputs were stored in reusable formats and verified through PostgreSQL, MinIO, DuckDB and finance reconciliation checks.

3. Technologies Used

Technology	Purpose
Python and Pandas	Data ingestion, cleaning, transformation and report generation
Docker	Running PostgreSQL and MinIO locally
PostgreSQL 16	Relational storage for dimensional tables
MinIO	Object storage for project outputs and data lake files
DuckDB	Analytical queries and local OLAP processing
CSV and Parquet	Portable and analytical data storage formats
Star schema	Organizing dimensional data for analysis

4. System Architecture

The workflow follows a local modern data stack pattern. Raw files are processed using Python, saved as cleaned and partitioned datasets, analyzed with DuckDB, exported to PostgreSQL and uploaded to MinIO for

object storage.

Layer	Component	Function
Input	Raw sales files	Source transaction data
Processing	Python and Pandas	Cleaning, normalization, deduplication and calculations
Object storage	MinIO	Stores output, reports and star schema files
Relational storage	PostgreSQL	Stores stores, products, categories and price revisions
Analytics	DuckDB	Runs analytical queries over local data and Parquet
Output	CSV, Parquet and DuckDB	Reusable analytical and reporting datasets

5. Completion of Project Stages A–F

The completed pipeline covers the six major stages below. These names summarize the work performed; the official assignment labels should be retained if they use different wording.

Stage	Completed work	Status
A. Ingestion	Raw files were read and combined.	Completed
B. Cleaning	Fields were normalized, records cleaned and duplicates removed.	Completed
C. Storage	Cleaned, partitioned, CSV and Parquet outputs were generated and uploaded to MinIO.	Completed
D. Analytics	DuckDB and daily/monthly analytical reports were generated.	Completed
E. Modeling	Dimensional data and star schema outputs were created and exported to PostgreSQL.	Completed
F. Validation	PostgreSQL counts, MinIO contents and finance reconciliation were verified.	Completed

6. Data Ingestion and Cleaning

The pipeline processed 1,137,585 raw rows and removed 16,661 duplicate records. The data processing stage produced cleaned sales data and transformed fields required for reporting, database export and analytical queries. The cleaned sales dataset was saved as `sales_cleaned.parquet`.

The generated outputs included `daily_sales.csv`, `monthly_sales.csv`, `sales_cleaned.parquet`, stores files, products files, product category files and price revision files in CSV and Parquet formats.

7. MinIO Object Storage

The MinIO bucket named `annapurna` was verified through the MinIO Object Browser. The bucket was private and displayed 172 objects using approximately 141.7 MiB of storage. The visible folders were `output`, `reports` and `star_schema`.

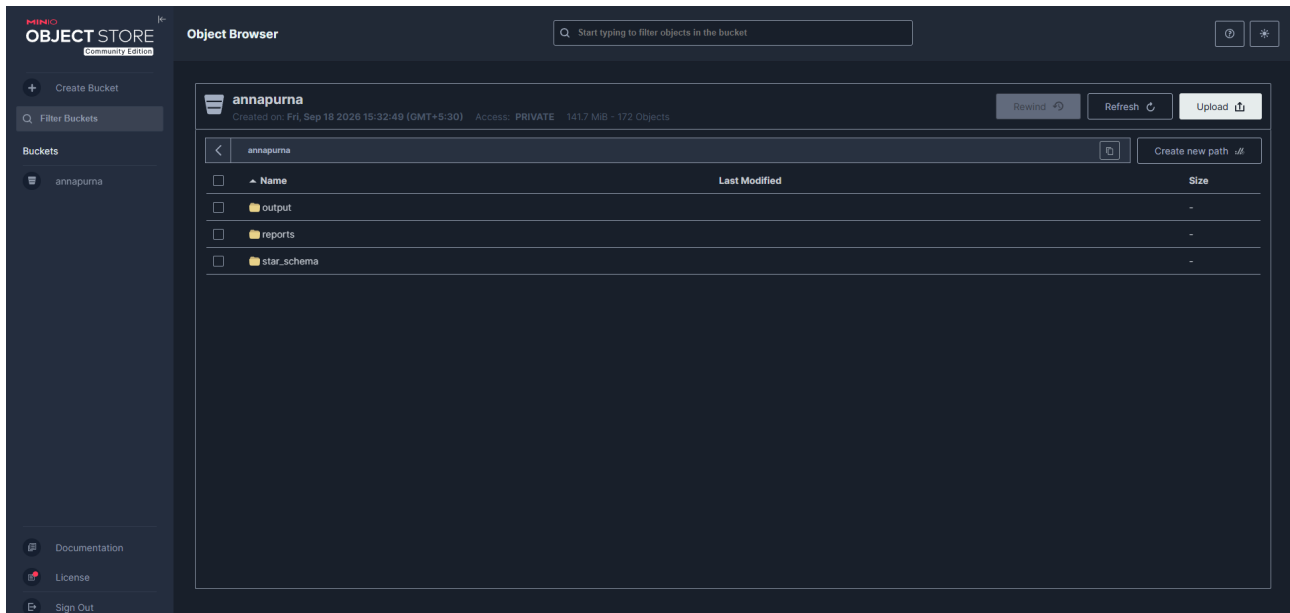


Figure 1. MiniIO bucket annapurna showing uploaded folders, object count and storage usage.

8. PostgreSQL Database

The correct PostgreSQL container was verified as annapurna_postgres. The database annapurna was accessed using the PostgreSQL command-line client. The database contained four exported tables.

Table	Verified row count
stores	12
product_categories	14
products	1,224
price_revisions	4,320

These counts confirm that the PostgreSQL export was completed and that the expected dimensional tables were available.

9. DuckDB Analytics

The pipeline generated annapurna.duckdb. DuckDB was used for analytical processing and aggregation over the cleaned data. Daily and monthly sales reports were also generated.

Output	Description
annapurna.duckdb	Local analytical database
daily_sales.csv	Daily sales aggregation
monthly_sales.csv	Monthly sales aggregation
partitioned/	Partitioned analytical data

10. ELT, Star Schema and Parquet Storage

The pipeline created dimensional outputs for stores, product categories, products and price revisions. These outputs were stored in CSV and Parquet formats and exported to PostgreSQL. The star_schema and

partitioned folders were included in the final submission folder.

Parquet was used as a columnar format suitable for analytical workloads. Partitioned data was generated to organize the data for analytical access.

11. Finance Reconciliation

Finance reconciliation was completed and the report was saved as `finance_reconciliation.csv`. The generated file contains calculated revenue for all twelve months of 2024.

Month	Calculated revenue (INR)
2024-01	41,629,544.81
2024-02	37,539,932.03
2024-03	45,168,740.99
2024-04	40,986,594.69
2024-05	44,865,035.18
2024-06	42,052,907.32
2024-07	43,292,272.79
2024-08	48,690,619.05
2024-09	48,161,223.24
2024-10	60,718,437.18
2024-11	55,888,183.67
2024-12	54,640,017.72

```
annapurna=# \q
What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug annapurna\_postgre
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\ub02-glab-012\Downloads\Q1_Annapurna> Get-Content .\reports\finance_reconciliation.csv
month,calculated_revenue
2024-01,41629544.81
2024-02,37539932.03
2024-03,45168740.99
2024-04,40986594.69
2024-05,44865035.18
2024-06,42052907.32
2024-07,43292272.79
2024-08,48690619.05
2024-09,48161223.24
2024-10,60718437.18
2024-11,55888183.67
2024-12,54640017.72
PS C:\Users\ub02-glab-012\Downloads\Q1_Annapurna> |
```

Figure 2. PowerShell output showing the generated `finance_reconciliation.csv` file.

12. Generated Files and Final Submission

The output directory contained the DuckDB database, daily and monthly reports, cleaned sales Parquet data, dimension files, partitioned data and star schema data. These files were copied into the `final_submission` folder.

Directory: C:\Users\ub02-glab-012\Downloads\Q1_Annapurna\output				
Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	18-09-2026	15:27		partitioned
d-----	18-09-2026	15:27		star_schema
-a-----	18-09-2026	15:27	274432	annapurna.duckdb
-a-----	18-09-2026	15:32	11708	daily_sales.csv
-a-----	18-09-2026	15:32	430	monthly_sales.csv
-a-----	18-09-2026	15:27	209791	price_revisions.csv
-a-----	18-09-2026	15:27	102611	price_revisions.parquet
-a-----	18-09-2026	15:27	106629	products.csv
-a-----	18-09-2026	15:27	32616	products.parquet
-a-----	18-09-2026	15:27	472	product_categories.csv
-a-----	18-09-2026	15:27	3196	product_categories.parquet
-a-----	18-09-2026	15:32	21509063	sales_cleaned.parquet
-a-----	18-09-2026	15:27	1299	stores.csv
-a-----	18-09-2026	15:27	6044	stores.parquet

Figure 3. Generated files in the output directory.

Directory: C:\Users\ub02-glab-012\Downloads\Q1_Annapurna\final_submission				
Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	18-09-2026	15:43		partitioned
d-----	18-09-2026	15:43		star_schema
-a-----	18-09-2026	15:27	274432	annapurna.duckdb
-a-----	18-09-2026	15:32	11708	daily_sales.csv
-a-----	18-09-2026	15:32	278	finance_reconciliation.csv
-a-----	18-09-2026	15:32	430	monthly_sales.csv
-a-----	18-09-2026	15:27	209791	price_revisions.csv
-a-----	18-09-2026	15:27	106629	products.csv
-a-----	18-09-2026	15:27	472	product_categories.csv
-a-----	18-09-2026	15:32	21509063	sales_cleaned.parquet
-a-----	18-09-2026	15:27	1299	stores.csv

Figure 4. Final submission folder containing the project files, partitioned data and star schema.

13. Conclusion

The Annapurna modern data stack implementation was completed successfully. The pipeline performed data processing, generated analytical files, created a DuckDB database, exported dimensional data to PostgreSQL, uploaded outputs to MinIO and completed finance reconciliation. The final submission folder contains the main data files, analytical database, partitioned data and star schema outputs required for the project.

The project is ready for submission with the generated files and the evidence screenshots included in this report.